UNIVERSIDAD DE DAGUPAN

SCHOOL OF INFORMATION TECHNOLOGY EDUCATION

ITP10 | EVENT-DRIVEN PROGRAMMING

PRELIM | LABORATORY ACTIVITY 3

Name : De Guzman, Francine Nicole J
Year, Course & blk : 31 ITE 02
Subject : Event-Driven Programming
Date : AUG. 25 2026

SCORE:

Laboratory Activity #3: Control-Structure Application

Objective:

This activity aims to help students apply JavaScript variables, operators, conditions, a `for` loop, input validation, functions, DOM interaction, and basic input and output in creating a simple checkout application that can be tested automatically.

Activity Description

Create a browser-based JavaScript checkout application using `index.html` and `script.js`. The application must accept information about several products through an HTML interface, compute the customer's total purchase, apply the appropriate discount, add a delivery fee, validate inputs, and display a complete order summary.

Application Requirements

The program must:

1. Accept the customer's name through an input field with the ID `customerName` and the visible label/prompt "Customer Name".
2. Accept how many products will be purchased through an input field with the ID `productCount` and the visible label/prompt "Number of Products".
3. Use a `for` loop to dynamically create and process the following information `for` each product:
   - Product name — ID pattern: `productName-0`, `productName-1`, `productName-2`, ...
   - Price — ID pattern: `productPrice-0`, `productPrice-1`, `productPrice-2`, ...
   - Quantity — ID pattern: `productQuantity-0`, `productQuantity-1`, `productQuantity-2`, ...
4. Validate that the customer name is not empty and that the product count, price, and quantity are valid positive numbers. Display validation feedback in the element with ID `validationMessage`.
5. Compute the amount for each product by calling `calculateItemAmount(price, quantity)`:
   `Item Amount = Price × Quantity`
6. Add all item amounts to determine the subtotal.
7. Use conditional statements inside `calculateDiscount(subtotal)` to apply the following discount:

Subtotal	Discount
₱5,000 and above	10%
₱3,000–₱4,999.99	7%
₱1,000–₱2,999.99	5%
Below ₱1,000	No discount

8. Ask the user to select a delivery option using a select element with the ID `deliveryOption` and the visible label/prompt "Delivery Option".
9. Use a `switch` statement inside `getDeliveryFee(option)` to determine the delivery fee:

Option	Delivery Type	Fee
1	Store Pickup	₱0
2	Standard Delivery	₱80
3	Express Delivery	₱150

10. Compute the final amount:
    `Final Amount = Subtotal - Discount + Delivery Fee`

11. When the user clicks the button with ID `calculateBtn` ("Calculate Order"), display the complete order summary inside the element with ID `orderSummary`. `alert()` or `console.log()` may be used only as optional debugging output.

Required JavaScript Concepts

The program must demonstrate the proper use of:

- `let` and `const`
- Arithmetic and comparison operators
- Type conversion using `Number()` or `parseFloat()`
- `if`, `else if`, and `else`
- `switch` statement
- A `for` loop `for` product generation/processing (required; other loops may be used additionally)
- Input validation
- Template literals
- `DOM` input/output, event handling, and optional debugging with `alert()` or `console.log()`
- Accumulators

Autograder Compatibility Requirements

To ensure that the Programming Classroom can grade every submission consistently, students must use the exact filenames, function names, element IDs, and labels below. The visual design and CSS may still be customized.

- Required files: `index.html` and `script.js`.
- Required top-level function: `calculateItemAmount(price, quantity)` — returns `price * quantity`.
- Required top-level function: `calculateDiscount(subtotal)` — returns the discount amount according to the required discount brackets.
- Required top-level function: `getDeliveryFee(option)` — uses a `switch` statement and returns 0, 80, or 150.
- Required static IDs: `customerName`, `productCount`, `productsContainer`, `deliveryOption`, `calculateBtn`, `validationMessage`, and `orderSummary`.
- Required dynamic product ID pattern: `productName-N`, `productPrice-N`, and `productQuantity-N`, where N starts at 0.
- Required visible labels/prompts: "Customer Name", "Number of Products", "Product Name", "Price", "Quantity", "Delivery Option", and button text "Calculate Order".
- Use a `for` loop for product input generation and/or product processing so the loop requirement is consistent for automated checking.
- The three required calculation functions must return values and must not directly call `prompt()`, `alert()`, or manipulate the DOM. Keep input/output handling separate from calculation logic.
- The required HTML form is the primary interface. `prompt()` may be used for optional practice or debugging, but it must not replace the required HTML inputs.

Suggested Program Flow

1. Display the application title: MINI STORE CHECKOUT SYSTEM.
2. Accept the customer's name from `customerName`.
3. Accept the number of products from `productCount`.
4. Use a `for` loop to generate and/or process the required product input fields inside `productsContainer`.
    - Accept the product name, price, and quantity using the required dynamic ID pattern.
    - Validate the entered values and display errors in `validationMessage`.
    - Calculate each item amount using `calculateItemAmount(price, quantity)`.
    - Add each item amount to the subtotal using an accumulator.
    - Build the product details that will appear in `orderSummary`.
5. Determine the discount using `calculateDiscount(subtotal)`, implemented with `if...else if...else`.
6. Determine the delivery fee using `getDeliveryFee(option)`, implemented with a `switch` statement.
7. Calculate the final amount: Subtotal − Discount + Delivery Fee.
8. On `calculateBtn` click, display the complete summary in `orderSummary`.

Sample Output

```
MINI STORE CHECKOUT SYSTEM

Customer: Juan Dela Cruz

1. Keyboard
   Price: ₱850.00
   Quantity: 2
   Amount: ₱1,700.00

2. Mouse
   Price: ₱450.00
   Quantity: 1
   Amount: ₱450.00

ORDER SUMMARY
Subtotal: ₱2,150.00
Discount Rate: 5%
Discount Amount: ₱107.50
Delivery Type: Standard Delivery
Delivery Fee: ₱80.00
Final Amount: ₱2,122.50
```

Procedure

1. Create a folder named `MiniStoreCheckout`.
2. Create the following files inside the folder:
   - `index.html`
   - `script.js`
3. Connect `script.js` to `index.html` using a `script` tag.
4. In `index.html`, create the checkout interface using the exact required static IDs and visible labels/prompts.
5. In `script.js`, define the required top-level functions exactly as named: `calculateItemAmount()`, `calculateDiscount()`, and `getDeliveryFee()`.
6. Use a `for` loop to generate and/or process product inputs, validate the values, compute the subtotal, and build the order summary.
7. Attach the calculation to `calculateBtn` and test all discount ranges, all delivery options, multiple products, and invalid inputs.
8. Correct any syntax, logic, runtime, validation, or browser-console errors before submission.

Scoring Rubric

Criteria	Excellent	Very Good	Satisfactory	Needs Improvement	Points
Program Functionality	All required interface elements, functions, calculations, validation, and checkout behaviors work correctly.	Most features work with minor errors.	Some features work, but several errors are present.	The application is incomplete or does not function properly.	30
Use of Control Structures	The required for loop, if/else if/else conditions, and switch statement are used correctly and appropriately.	Control structures are mostly correct with minor errors.	Control structures are present but contain several errors.	Required control structures are missing or incorrectly used.	25
Input Validation and Computation	All required inputs are validated, and item amount, subtotal, discount, delivery fee, and final amount are accurate.	Most inputs and calculations are correct.	Validation is limited or some calculations are incorrect.	Inputs are not validated, and calculations are mostly incorrect.	20
Output and Presentation	The required labels/prompts and order summary are complete, clear, organized, and properly formatted.	The output is understandable with minor formatting issues.	The output is incomplete or poorly organized.	The output is unclear or mostly missing.	15
Code Quality	Code is readable and properly indented, and the required filenames, function names, and element IDs are followed exactly.	Code is generally readable with minor issues.	Code organization and readability need improvement.	Code is difficult to read and poorly organized.	10
Total					**100**

PREPARED BY:

ROCHELLE S. LANTO, MIT & JELSON V. LANTO, MIT
SITE, FACULTY